

IPTABLES

<https://linux-man.fr/index.php/2021/09/09/iptables/>

https://www.youtube.com/watch?v=ByTtN3pgnP0&list=PLn6POgpkWwrl_ri_djf3R3RCyDOsyKoC

Créer 2 machines linux à partir du modèle ➔ SIO1-Modele-B3-Iptable-Debian1. **Fait.**

Nous nommerons ces machines iptable1 et iptables2,

Réseau 172.16.10.0/24

Rappel, afin de paramétrer les adresses ip ==> modifier le fichier /etc/network/interfaces

```
1 # This file describes the network interfaces available on your system
2 # and how to activate them. For more information, see interfaces(5).
3
4 source /etc/network/interfaces.d/*
5
6 # The loopback network interface
7 auto lo ens3
8 iface lo inet loopback
9
10 auto ens3
11 iface ens3 inet static
12 address 172.16.10.112
13 netmask 255.255.255.0
14 gateway 172.16.10.254
15 dns|nameservers 172.16.10.100, 8.8.8.8
```

SISR

Akram 201-203	Erwann 204-207	Maxence 208-210	Lenny 211-213
Kilian 214-216	Théo 217-219	Yanis 220-222	Rochdi 223-225
Omar 226-228	Ryiad 229-231	Oguzhan 232-234	Valentin 235-237
Gaëtan 238-240	Nathan 241-243		

SLAM

Ilyas 101-103	Christian 104-106	Marwan 107-109	Thibault 110-112
Diego 113-115	Margerit 116-118	Tracy 119-121	Antoine 122-124
Lucas 125-127	Ryiad 128-130	Clément 131-133	Gaëtan 134-136
Kevin 137-139	Mikailcan 140-142	Yannis 143-145	Yliasse 146-148

Trouver le numéro d'une règle :

```
root@debian:/home/eleve# iptables -L --line-numbers
Chain INPUT (policy ACCEPT)
num target      prot opt source                destination          reject-with i
1 REJECT        icmp -- anywhere             anywhere
cmp-port-unreachable

Chain FORWARD (policy ACCEPT)
num target      prot opt source                destination

Chain OUTPUT (policy ACCEPT)
num target      prot opt source                destination
```

Supprimer une règle :

```
root@debian:/home/eleve# iptables -D INPUT 1
root@debian:/home/eleve# iptables -L --line-numbers
Chain INPUT (policy ACCEPT)
num target      prot opt source                destination

Chain FORWARD (policy ACCEPT)
num target      prot opt source                destination

Chain OUTPUT (policy ACCEPT)
num target      prot opt source                destination
```

Exercice 2 :

Utilisation du module python qui simule l'ouverture d'un serveur http sur un port donné.

Ce module est installé avec la version de Debian.



```
root@debian:/test# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Il s'agit de lancer ce « serveur » sur une machine *adr1* puis de se connecter à partir de l'autre.

➔ici *adr1* a pour valeur 172.16.10.112 (exemple, il faut prendre les adresses qui vous sont attribuées !)

```
root@debian:/home/eleve# telnet 172.16.10.112 8000
Trying 172.16.10.112...
Connected to 172.16.10.112.
```

a) Refuser toutes les connections au port 8000 à cette machine *adr1*. **Fait.**

```
root@debian:/home/eleve# iptables -I INPUT -p tcp --dport 8000 -j REJECT
```

Tester que le résultat est conforme à l'attendu (aucune machine ne peut se connecter à ce port).

Fait.

b) Accepter les connections d'une machine tiers (seulement celle-ci). **Fait.**

```
root@debian:/home/eleve# iptables ... |
```

à compléter

Exercice 3 :

Installer open-ssh sur la machine iptable1. **Fait.**

Tester la connexion de la machine iptable2 vers iptable1,

Ajouter des règles de filtre afin que seule la machine iptable2 ait le droit se connecter à iptable1 en ssh.

Autoriser le camarade avec qui vous partager la même rangée à se connecter en ssh.

Ecrire ici les commandes à effectuer.

Tester votre filtre (une autre personne ne doit pas pouvoir se connecter)

Exercice 4 :

Les règles Iptables que nous avons créées sont enregistrées en mémoire. Cela signifie que nous devons les redéfinir au redémarrage. Pour que ces modifications soient persistantes après le redémarrage, utilisez la commande suivante sur les systèmes Ubuntu/Debian:

```
sudo /sbin/iptables-save
```

Cette commande sauve les règles actuelles dans le fichier de configuration du système qui est utilisé pour reconfigurer les tableaux au moment du redémarrage. Vous devez exécuter cette commande chaque fois que vous modifiez les règles. Pour désactiver ce pare-feu, il suffit de supprimer toutes les règles et de rendre les modifications persistantes.

```
sudo iptables -F  
sudo /sbin/iptables-save
```

Objectif

➤ Résoudre les problèmes de sécurité associés à la communication en réseau

Contexte

Le cabinet MAURIGGIA Architecture est composé de deux agences, la première est située à Fresnes, la deuxième à Cannes. Les employés de ces deux agences doivent pouvoir échanger des documents en toute sécurité.

Ces échanges doivent être confidentiels, le destinataire du document doit être certain que le message n'a pas été modifié entre l'émission et la réception et enfin les participants doivent être authentifiés.

1^{ère} partie : Chiffrer des fichiers

1.1. Créer le fichier texte `clair.txt` avec le texte suivant : "Bonjour".

Chiffrer ce fichier avec `openssl` en utilisant l'algorithme RC4, le mot de passe utilisé sera "secret".

```
touch clair.txt//nano clair.txt//openssl aes-256-cbc -k secret -in clair.txt -out test5.dat
```

1.2. Le fichier d'origine `clair.txt` n'est plus nécessaire, supprimez le.

Visualiser le fichier chiffré. Comment est le contenu de ce fichier ?

```
rm clair.txt//cat test5.dat// Le contenu de ce fichier est chiffré.
```

1.3. Déchiffrer le fichier chiffré précédemment.

Visualiser le contenu du fichier `clair.txt`.

```
openssl aes-256-cbc -d -k secret <test5.dat> clair.txt//cat clair.txt
```

1.4. Déchiffrer de nouveau le fichier chiffré précédemment dans le fichier `clair2.txt` en indiquant le mot de passe en argument et en utilisant la redirection des entrées-sorties standards.

Visualiser le contenu du fichier `clair2.txt`.

```
openssl aes-256-cbc -d -k secret <test5.dat> clair2.txt//cat clair2.txt
```

1.5. En utilisant la commande `vim`, créer le fichier `protégé.txt` avec le mot de passe `azerty`.

Visualiser le contenu de ce fichier avec la commande `cat`.

```
Je ne peux pas le faire, n'ai pas la commande vim de disponible, et ne peux pas l'installer car pas d'internet.
```

1.6. Faites en sorte de pouvoir visualiser clairement le contenu du fichier `protégé.txt`.

```
Je ne peux pas le faire, n'ai pas la commande vim de disponible, et ne peux pas l'installer car pas d'internet.
```

2^{ème} partie : Les générateurs d'empreintes

2.1. Générer les empreintes MD5, SHA-1 et SHA-256 du fichier `clair.txt` avec `openssl`.

```
openssl dgst -md5 <clair.txt> test5.emp
openssl dgst -sha1 <clair.txt> test5.emp
openssl dgst -sha256 <clair.txt> test5.emp
```

2.2. Quelle est la taille de l'empreinte générée par MD5, SHA1 et SHA256 ? Conclure.

```
Taille de l'empreinte générée par MD5 : 128 Bits
Taille de l'empreinte générée par SHA1 : 160 Bits
Taille de l'empreinte générée par SHA256 : 256 Bits

Donc, on peut en conclure que la sécurité de SHA256 > SHA1 > MD5.
```

2.3. On va maintenant calculer l'empreinte de deux textes très proches.

Exécuter la commande : `sed '1s/^b/c/' clair.txt > clair2.txt`

Que fait cette commande ? Détailler.

```
J'ai exécuter la commande sed -i -e 's/b/c/g' clair2.txt.
Cette commande sert à manipuler les informations du fichier clair2.txt.
La lettre 'b' du fichier a été remplacer par la lettre 'c'.
```

2.4. Générer les empreintes MD5 de chaque fichier.

Comparer avec leurs empreintes.

```
openssl md5 clair.txt
les empreintes sont différentes
```

2.5. En utilisant les commandes `sum`, `md5sum` et `shasum`, calculer la somme de contrôle d'une empreinte pour pouvoir vérifier son intégrité par le récepteur du fichier `clair.txt`.

```
sum clair.txt
md5sum clair.txt
shasum clair.txt
```

2.6. Générer la somme de contrôle MD5 du fichier `clair.txt` (à sauvegarder dans le fichier `clair.md5`), modifier le fichier d'origine `clair.txt` puis vérifier la somme de contrôle de ce fichier.

Commenter.

Création du fichier `clair.md5` :

Vérification de la somme de contrôle du fichier `clair.txt` :

Modification du fichier `clair.txt`.

Vérification de la somme de contrôle du fichier `clair.txt` :

Commenter :

3^{ème} partie : Le chiffrement symétrique

3.1. Visualiser les algorithmes de chiffrement symétrique supportés par votre distribution à l'aide de la commande `man enc`.

Utilisation de l'algorithme RC4 :

xChiffrement avec RC4 : `openssl rc4 -e -in clair.txt -out crypto.dat -k la_cle`

xDéchiffrement : `openssl rc4 -d -in crypto.dat -k la_cle`

3.2. Utiliser les méthodes de cryptage symétriques suivantes : RC4, DES, 3DES, AES, RC2. Lancer les commandes correspondantes et pour chaque chiffrement, sauvegarder le résultat dans un fichier.

```
→openssl rc4 -e -in clair.txt -out crypto.dat -k la_cle fonctionne
→openssl DES -e -in clair.txt -out crypto.dat -k la_cle fonctionne
→openssl 3DES -e -in clair.txt -out crypto.dat -k la_cle ne fonctionne pas
→openssl AES -e -in clair.txt -out crypto.dat -k la_cle ne fonctionne pas
→openssl RC2 -e -in clair.txt -out crypto.dat -k la_cle fonctionne
```

3.3. Visualiser les fichiers chiffrés.

3.4. Vérifier qu'il est possible de déchiffrer chaque fichier.

```
→openssl rc4 -d -in crypto.dat -k la_cle fonctionne
→openssl des -d -in crypto.dat -k la_cle fonctionne
→openssl 3des -d -in crypto.dat -k la_cle ne fonctionne pas
→openssl aes -d -in crypto.dat -k la_cle ne fonctionne pas
→openssl rc2 -d -in crypto.dat -k la_cle fonctionne
```

Utilisation du DES selon les différents modes de chiffrement par blocs.

3.5. Utilisation du mode CBC (Cipher Bloc Chaining). Chaque bloc clair est combiné par un XOR avec le bloc chiffré précédent avant d'être chiffré. Pour le premier bloc, on utilise un bloc dit vecteur d'initialisation (iv), choisi aléatoirement et qui est inscrit en clair dans le message crypté.

```
openssl des-cbc -e -in clair.txt -out crypto.dat -k la_cle -p
salt =
key =
iv =
```

3.6. Utilisation du mode ECB (Electronic Code Block). Chaque bloc est toujours codé avec la même clé. Donc, deux blocs identiques seront codés de la même manière.

3.7. Utilisation de CFB (Cipher Feed Back). La clé change à chaque bloc. Un octet du texte clair est combiné par un XOR avec un octet de la clé.

Chiffrer un message en DES, le système choisit une graine aléatoire :

```
openssl des -e -in clair.txt -out crypto3.dat -k la_cle -p
```

Faire la même chose mais en utilisant la graine précédente (salt) :

```
openssl des -e -in clair.txt -out crypto4.dat -k la_cle -S xxxxx  
(xxxx est à remplacer par la valeur du Salt obtenu précédemment)
```

3.8. Comparer les deux fichiers. Que remarquez-vous ?

3.9. Lancer la commande qui vous permet de chiffrer sans la graine, le logiciel choisit un vecteur d'initialisation aléatoire.

3.10. Chiffrer en utilisant le même vecteur d'initialisation que celui généré à la question précédente.

3.11. Comparer les deux fichiers chiffrés. Que remarquez-vous ?

4^{ème} partie : Le chiffrement asymétrique

Dans cette partie Pierre LASSAIRE (plassaire) doit transmettre un rapport à Timothée SARLAT (tsarlat) qui est le responsable de l'agence Mauriggia Architecture de Cannes.

Utilisation de RSA

4.1. En tant que Pierre LASSAIRE, générer un couple de clé (publique et privée).

4.2. Pierre LASSAIRE extrait la clé publique et la donne à Timothée SARLAT.

4.3. Timothée SARLAT chiffre un mot de passe en utilisant la clé publique de Pierre LASSAIRE et lui envoie le message.

4.4. Pierre LASSAIRE déchiffre le message avec sa clé privée.

La signature numérique

4.5. Pierre LASSAIRE calcule l'empreinte MD5 d'un fichier nommé rapport.txt et le signe avec sa clé privée.

4.6. Timothée SARLAT reçoit le fichier et la signature. Il recalcule l'empreinte.

- 4.7. Il déchiffre la signature avec la clé publique de Pierre LASSAIRE.
Les deux empreintes (recalculées et déchiffrées) sont-elles identiques ?